

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import json
import sys
import matplotlib.pyplot
import scipy.linalg

import plotly
import plotly.graph_objs as go
```

Introduction to numerical methods

János Török

Class 4: Linear algebra 3.

System of linear equations

The general form is:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m \end{cases}$$

where $x_1, x_2, \dots, x_n$ are the unknowns, $a_{11}, a_{12}, \dots, a_{mn}$ are the coefficients of the system, and $b_1, b_2, \dots, b_m$ are the constant terms.

In matrix form

$$Ax = b,$$

where

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}, x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}.$$

Gauss elimination

First we create a matrix $(A \vee b)$. Then we try to make the value below the diagonal zero, and the diagonal one. Allowed steps:

- Swapping two rows,
- Multiplying a row by a nonzero number,
- Adding a multiple of one row to another row.

Example:

$$\begin{pmatrix} 1 & 3 & 1 & 9 \\ 1 & 1 & -1 & 1 \\ 3 & 11 & 5 & 35 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 3 & 1 & 9 \\ 0 & -2 & -2 & -8 \\ 0 & 2 & 2 & 8 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 3 & 1 & 9 \\ 0 & -2 & -2 & -8 \\ 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & -2 & -3 \\ 0 & 1 & 1 & 4 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

The steps:

1. The first row times -1 is added to the second row, and the -3 times the first row is added to the third row
2. The second row is added to the third row resulting in a row with only zeros, which is a singular case.
3. The second row is multiplied by -2 and it is added to the first row multiplied by -3 .

Implementation

```
A = np.array([[1,3,1],[1,1,-1],[3,11,5]],dtype=float)
b = np.array([9,1,35],dtype=float)
```

```
B = np.column_stack((A,b))
print(B)
```

```
[[ 1.  3.  1.  9.]
 [ 1.  1. -1.  1.]
 [ 3. 11.  5. 35.]]
```

```
def GE_swap(B,i,j):
    tmp = np.copy(B[i])
    B[i] = np.copy(B[j])
    B[j] = tmp
```

```
GE_swap(B,1,2)
print(B)
B = np.column_stack((A,b))
```

```
[[ 1.  3.  1.  9.]
 [ 3. 11.  5. 35.]
 [ 1.  1. -1.  1.]]
```

```
def GE_multiply(B,i,alpha):
    B[i] = B[i] * alpha
```

```
GE_multiply(B,0,0.5)
print(B)
B = np.column_stack((A,b))
```

```

[[ 0.5  1.5  0.5  4.5]
 [ 1.   1.  -1.   1. ]
 [ 3.  11.   5.  35. ]]

def GE_add_multiply(B,i,j,alpha):
    B[i] += B[j] * alpha

GE_add_multiply(B,1,0,-1)
print(B)

[[ 1.   3.   1.   9.]
 [ 0.  -2.  -2.  -8.]
 [ 3.  11.   5.  35.]]

```

Gauss elimination algorithm

1. Iterate for each column, except for the last (i)
2. In column i find an element in rows at or below i which are nonzero, and swap the rows if necessary
3. Clear all elements in column i below i In the end we should end up with a bottom triangle which is zero

```

#A safe point where we recreate the matrix and start our development
B = np.column_stack((A,b))
#GE_add_multiply(B,0,1,1)
#GE_swap(B,0,2)
print(B)
N = len(B)

[[ 1.   3.   1.   9.]
 [ 1.   1.  -1.   1.]
 [ 3.  11.   5.  35.]]

#column i from row i
i = 0
B[i:,i]

array([1., 1., 3.])

np.where(B[i:,i] > 0)[0]

array([0, 1, 2])

#Check the topmost row where there are nonzero elements, and give an error message if there aren't any
if len(np.where(B[i:,i] > 0)[0]) == 0:
    print("Singular matrix")
else:
    j = np.where(B[i:,i] > 0)[0].min()
    print(j)

0

```

```

#if the nonzero is in an other row, swap the two
if i != j:
    GE_swap(B,i,j)
print(B)

[[ 1.  3.  1.  9.]
 [ 1.  1. -1.  1.]
 [ 3. 11.  5. 35.]]

####
#Now the Gauss-elimination
####
#make the killer element 1
GE_multiply(B,i,1.0/B[i,i])
for j in range(i+1,N):
    GE_add_multiply(B,j,i,-B[j,i])
print(B)

[[ 1.  3.  1.  9.]
 [ 0. -2. -2. -8.]
 [ 0.  2.  2.  8.]]

#Solution
# i in reverse order
#left hand side

```

Solution

$$B = \begin{pmatrix} 1 & 3 & 1 & 9 \\ 0 & 1 & 1 & 4 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

Expanded form:

$$\begin{aligned} x_3 &= 1 \\ x_2 &= 4 - 1 \cdot x_3 \\ x_1 &= 9 - 3 \cdot x_2 - 1 \cdot x_3 \end{aligned}$$

Solution is

$$x_i = b_i - \sum_{j=i+1}^N b_j x_j$$

If the vector of solution at the beginning is zero $x = (x_1, x_2, x_3) = (0, 0, 0)$, then we can rewrite it as:

$$x_i = b_i - b x,$$

provided we go with i in the reverse order.

```

def my_GE(A,b):
    B = np.array(np.column_stack((A,b)),dtype=float)
    N = len(B)
    for i in range(N):
        if B[i,i] == 0.0:
            if len(np.where(B[i:,i] > 0)[0]) == 0:
                raise Exception("Singular matrix")
            else:
                j = np.where(B[i:,i] > 0)[0].min()
                GE_swap(B,i,j)
                GE_multiply(B,i,1.0/B[i,i])
            for j in range(i+1,N):
                GE_add_multiply(B,j,i,-B[j,i])
    sol = np.zeros(N,dtype=float)
    for i in reversed(range(N)):
        sol[i] = B[i,-1] - (np.dot(B[i,:N],sol)).sum()
    return sol

```

```

A = np.array([[1,3,1],[1,1,-1],[3,11,5]],dtype=float)
b = np.array([9,1,35],dtype=float)
my_GE(A,b)

```

```

-----
Exception                                Traceback (most recent call
last)

```

```

Cell In[19], line 3
      1 A = np.array([[1,3,1],[1,1,-1],[3,11,5]],dtype=float)
      2 b = np.array([9,1,35],dtype=float)
----> 3 my_GE(A,b)

```

```

Cell In[18], line 7, in my_GE(A, b)
      5 if B[i,i] == 0.0:
      6     if len(np.where(B[i:,i] > 0)[0]) == 0:
----> 7         raise Exception("Singular matrix")
      8     else:
      9         j = np.where(B[i:,i] > 0)[0].min()

```

```

Exception: Singular matrix

```

```

A = np.array([[1,3,1],[1,1,-1],[3,10,5]],dtype=float)
b = np.array([9,1,32],dtype=float)
x = my_GE(A,b)
print(x)

```

```

[-1.  3.  1.]

```

Iterative methods: Gauss-Seidel

Sometimes iterative methods are desirable. Especially if there are limit issues, e.g. there are too small and too large variables, or if e.g. the matrix is a result of a dynamical process and changes slowly and so the solutions, than iterative methods can be much faster. An other important use case is the sparse matrices, where creating an algorithm which uses this nature to its advantage is easy with iterative methods but impossible with Gauss elimination.

Let us express the components of x from the set of linear equations in an iterative form, namely supposing we know all other components:

$$Ax=b$$

In components:

$$\begin{array}{lcl} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n & \stackrel{!}{=} & b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n & \stackrel{!}{=} & b_2 \\ \vdots & \stackrel{!}{=} & \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n & \stackrel{!}{=} & b_n \end{array}$$

The different components can be expressed using superscript for iteration count:

$$\begin{array}{lcl} x_1^{(i+1)} & \stackrel{!}{=} & \frac{-1}{a_{11}} (a_{12}x_2^{(i)} + \dots + a_{1n}x_n^{(i)} - b_1) \\ x_2^{(i+1)} & \stackrel{!}{=} & \frac{-1}{a_{22}} (a_{21}x_1^{(i)} + a_{23}x_3^{(i)} + \dots + a_{2n}x_n^{(i)} - b_2) \\ \vdots & \stackrel{!}{=} & \vdots \\ x_n^{(i+1)} & \stackrel{!}{=} & \frac{-1}{a_{nn}} (a_{n1}x_1^{(i)} + a_{n2}x_2^{(i)} + \dots + a_{n,n-1}x_{n-1}^{(i)} - b_n) \end{array}$$

Please note the minus sign in the nominator of the first term on the right hand side of the equation.

This is the Jordan method, so what can we do better? Why not to use the already calculated values. It was shown that it converges faster.

$$\begin{array}{lcl} x_1^{(i+1)} & \stackrel{!}{=} & \frac{-1}{a_{11}} (a_{12}x_2^{(i)} + \dots + a_{1n}x_n^{(i)} - b_1) \\ x_2^{(i+1)} & \stackrel{!}{=} & \frac{-1}{a_{22}} (a_{21}x_1^{(i+1)} + a_{23}x_3^{(i)} + \dots + a_{2n}x_n^{(i)} - b_2) \\ \vdots & \stackrel{!}{=} & \vdots \\ x_n^{(i+1)} & \stackrel{!}{=} & \frac{-1}{a_{nn}} (a_{n1}x_1^{(i+1)} + a_{n2}x_2^{(i+1)} + \dots + a_{n,n-1}x_{n-1}^{(i+1)} - b_n) \end{array}$$

In form of a sum:

$$x_j^{(i+1)} = \frac{-1}{a_{jj}} \left(\sum_{k=1}^{j-1} a_{jk} x_k^{(i+1)} + \sum_{k=j+1}^n a_{jk} x_k^{(i)} - b_j \right)$$

The last but most important thing to mention is that this method converges only if the matrix A is either diagonally dominant, positive definite, or symmetric. Let us first write the algorithm keeping in mind that all iterative methods need starting values.

```
#once again work step by step
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)
b = np.array([1,32,-3],dtype=float)
N = len(b)
my_GE(A,b)

array([-1.,  3.,  1.])

#initial values
x0 = np.ones(N,dtype=float)
x = np.copy(x0)

#iteration, here it is very simple
for j in range(N):
    x[j] = -(np.sum(A[j] * x) - A[j,j] * x[j] - b[j]) / A[j,j]
    print(x)

[-0.33333333  1.          1.          ]
[-0.33333333  2.8         1.          ]
[-0.33333333  2.8         0.13333333]

# stopping conditions
prev_x = np.copy(x)
for j in range(N):
    x[j] = -(np.sum(A[j] * x) - A[j,j] * x[j] - b[j]) / A[j,j]
print(np.linalg.norm(x-prev_x))

1.0373232409296727

#Besides A and b we need three parameters, the stopping limit, maximum
iteration and the initial condition
def my_GS(A, b, lim = 1e-3, max_iter = 1000, x0 = []):
    #if init is not given we will start from the vector (1,1,...,1)
    N = len(b)
    if len(x0) != N:
        x0 = np.ones(N,dtype=float)
    x = np.copy(x0)
    for i in range(max_iter):
        prev_x = np.copy(x)
        for j in range(N):
            x[j] = -(np.sum(A[j] * x) - A[j,j] * x[j] - b[j]) / A[j,j]
        if np.linalg.norm(x-prev_x) < lim:
            break
    return x,i,np.linalg.norm(x-prev_x)
```

```

#once again work step by step
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)
b = np.array([1,32,-3],dtype=float)

x, niter,lim = my_GS(A,b)
print(x)
print(niter,lim)

[-0.99967631  3.00021519  0.9998915 ]
9 0.0007641281018185905

%%timeit
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)
b = np.array([1,32,-3],dtype=float)
my_GE(A,b)

32.9 µs ± 748 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops
each)

%%timeit
A = np.array([[3,1,1],[3,10,5],[1,-1,1]],dtype=float)
b = np.array([1,32,-3],dtype=float)
my_GS(A,b)

193 µs ± 1.37 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops
each)

A = np.array([[1,3,1],[1,1,-1],[3,10,5]],dtype=float)
b = np.array([9,1,32],dtype=float)
x, niter,lim = my_GS(A,b)
print(x)
print(niter,lim)

[ 8.69234435e+191  1.87447450e+192 -4.27048967e+192]
999 inf

#symmetric matrix
A = np.array([[1,3,1],[1,4,2],[1,2,4]],dtype=float)
b = np.array([9,13,9],dtype=float)
x, niter,lim = my_GS(A,b)
print(x)
print(niter,lim)

[-0.99705189  2.99939451  0.99956572]
16 0.0003525996154096707

```

LU decomposition

The matrix A is decomposed into an L lower triangle and U upper triangle matrix such that $A=LU$. Example:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}$$

In particular a common choice is that:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{pmatrix},$$

in which case the decomposition is unique.

Simple solution:

Use Gauss elimination to get L , in a form $Lx = x$:

$$\begin{pmatrix} 1 & 0 & \dots & 0 \\ l_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & 0 \\ l_{n1} & l_{n2} & \dots & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

Such a matrix can be obtained by writing a Gauss elimination algorithm upside down. The U matrix can be obtained by performing the matrix multiplication of LU and going through in a typewriter style:

The first line:

$$\begin{array}{l} a_{11} \quad \textcolor{red}{l}1 \cdot u_{11} \\ a_{12} \quad \textcolor{red}{l}1 \cdot u_{12} \\ \vdots \quad \textcolor{red}{l}\vdots \\ a_{1n} \quad \textcolor{red}{l}1 \cdot u_{1n} \end{array}$$

So the first line is the same as that of A . The second line:

$$\begin{array}{l} a_{21} \quad \textcolor{red}{l}l_{21} \cdot u_{11} \\ a_{22} \quad \textcolor{red}{l}l_{21} \cdot u_{12} + l_{22} \cdot u_{22} \\ \vdots \quad \textcolor{red}{l}\vdots \\ a_{2n} \quad \textcolor{red}{l}\sum_j l_{2j} \cdot u_{jn} \end{array}$$

We can see that we only use parts of the U matrix which is already known.

The algorithm is not too difficult we could use our Gauss elimination but we will not implement it. The reason is the possibility of singularity during the process requires pivoting elements (changing rows and columns). One has to keep track of that in three matrices, in L , in the actually created U and a pivot matrix P , which is defined as $PA = UL$. In general the procedure is done in a different order. First the pivot matrix is determined from the maximum values of the A matrix. If only partial pivoting is done, then the one after the other the row with the highest

element in the diagonal is swapped to the actual position, in the case of full pivoting also columns are swapped. The pivoting procedure is coded in the pivot matrix.

```
def my_LU_decomposition_with_pivoting(A):  
    """  
        Perform LU decomposition with partial pivoting of a square matrix  
A.  
  
        The matrix A is decomposed into a lower triangular matrix L, an  
upper triangular matrix U,  
and a permutation matrix P such that  $PA = LU$ .  
  
        Parameters:  
A (numpy.ndarray): The square matrix to decompose.  
  
        Returns:  
P (numpy.ndarray): The permutation matrix.  
L (numpy.ndarray): The lower triangular matrix.  
U (numpy.ndarray): The upper triangular matrix.  
    """  
    n = A.shape[0]  
  
    # Initialize L as a zero matrix, U as a copy of A, and P as the  
identity matrix  
    L = np.zeros_like(A, dtype=float)  
    U = np.copy(A)  
    P = np.eye(n)  
  
    for i in range(n):  
        # Partial pivoting: find the row with the largest element in  
the current column  
        pivot = np.argmax(np.abs(U[i:, i])) + i  
  
        # Swap the rows in U, L, and P  
        if pivot != i:  
            U[[i, pivot]] = U[[pivot, i]]  
            P[[i, pivot]] = P[[pivot, i]]  
            if i > 0:  
                L[[i, pivot], :i] = L[[pivot, i], :i]  
  
        # Update L and U matrices  
        L[i, i] = 1  
        for j in range(i + 1, n):  
            L[j, i] = U[j, i] / U[i, i]  
            U[j, i:] -= L[j, i] * U[i, i:]  
        print("III", i)  
        print("PPP", P)  
        print("LLL", L)  
        print("UUU", U)  
        print("#####")
```

```

    return P, L, U
A = np.array([[3, -1, 4], [17, 2, 1], [1, 12, -7]])
print("Matrix A:")
print(A)
P, L, U = my_LU_decomposition_with_pivoting(A.astype(float))

print("\nPermutation matrix P:")
print(P)
print("\nLower triangular matrix L:")
print(L)
print("\nUpper triangular matrix U:")
print(U)

# Verification: P @ A = L @ U should be close to A
print("\nVerification: P @ A =")
print(np.dot(P, A))
print("\nL @ U =")
print(np.dot(L, U))

Matrix A:
[[ 3 -1  4]
 [17  2  1]
 [ 1 12 -7]]
III 0
PPP [[0.  1.  0.]
      [1.  0.  0.]
      [0.  0.  1.]]
LLL [[1.         0.         0.         ]
      [0.17647059 0.         0.         ]
      [0.05882353 0.         0.         ]]
UUU [[17.         2.         1.         ]
      [ 0.         -1.35294118  3.82352941]
      [ 0.         11.88235294 -7.05882353]]
#####
III 1
PPP [[0.  1.  0.]
      [0.  0.  1.]
      [1.  0.  0.]]
LLL [[ 1.         0.         0.         ]
      [ 0.05882353  1.         0.         ]
      [ 0.17647059 -0.11386139  0.         ]]
UUU [[17.         2.         1.         ]
      [ 0.         11.88235294 -7.05882353]
      [ 0.         0.         3.01980198]]
#####
III 2
PPP [[0.  1.  0.]
      [0.  0.  1.]
      [1.  0.  0.]]

```

```

LLL [[ 1.          0.          0.          ]
     [ 0.05882353  1.          0.          ]
     [ 0.17647059 -0.11386139  1.          ]]
UUU [[17.          2.          1.          ]
     [ 0.          11.88235294 -7.05882353]
     [ 0.           0.          3.01980198]]
#####

Permutation matrix P:
[[0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]]

Lower triangular matrix L:
[[ 1.          0.          0.          ]
 [ 0.05882353  1.          0.          ]
 [ 0.17647059 -0.11386139  1.          ]]

Upper triangular matrix U:
[[17.          2.          1.          ]
 [ 0.          11.88235294 -7.05882353]
 [ 0.           0.          3.01980198]]

Verification: P @ A =
[[17.  2.  1.]
 [ 1. 12. -7.]
 [ 3. -1.  4.]]

L @ U =
[[17.  2.  1.]
 [ 1. 12. -7.]
 [ 3. -1.  4.]]

```

Matrix inverse

$$(A^{-1})^{-1} = A$$

Or

$$A^{-1}A = A \cdot A^{-1} = I,$$

where I is the identity matrix.

First method: Gauss elimination!

Let us define: $e_x = (1, 0, 0)$, $e_y = (0, 1, 0)$, $e_z = (0, 0, 1)$. Let us search for the solution of:

$$Ax_1 = e_x.$$

If you think about matrix multiplication, x_1 is the first column of the inverse matrix. So in

$$Ax_2 = e_y, \text{ and } Ax_3 = e_z$$

x_2 and x_3 are the other two columns of the inverse matrix, so.

$$A \cdot (x_1, x_2, x_3) = (e_x, e_y, e_z)$$

```
#copy paste the GE and add the reverse pass
def my_GE_inverse(A):
    B = np.array(np.column_stack((A,np.identity(len(A)))),dtype=float)
    N = len(B)
    print(B)
    for i in range(N):
        if B[i,i] == 0.0:
            if len(np.where(B[i,:]>0)[0]) == 0:
                raise Exception("Singular matrix")
            else:
                j = np.where(B[i,:]>0)[0].min()
                GE_swap(B,i,j)
        GE_multiply(B,i,1.0/B[i,i])
        for j in range(i+1,N):
            GE_add_multiply(B,j,i,-B[j,i])
    #reverse pass
    for i in reversed(range(N)):
        for j in range(i):
            GE_add_multiply(B,j,i,-B[j,i])
    print(B)
    return B[:,3:]

A = np.array([[1,3,1],[1,1,-1],[3,10,5]],dtype=float)
np.linalg.inv(A)

array([[-7.5,  2.5,  2. ],
       [ 4. , -1. , -1. ],
       [-3.5,  0.5,  1. ]])

A1 = my_GE_inverse(A)

[[ 1.  3.  1.  1.  0.  0.]
 [ 1.  1. -1.  0.  1.  0.]
 [ 3. 10.  5.  0.  0.  1.]]
[[ 1.  0.  0. -7.5  2.5  2. ]
 [-0.  1.  0.  4.  -1. -1. ]
 [ 0.  0.  1. -3.5  0.5  1. ]]

print(A1)

[[-7.5  2.5  2. ]
 [ 4.  -1. -1. ]
 [-3.5  0.5  1. ]]
```

Using definition

$$A^{-1} = \frac{1}{\det(A)} \text{Adj}(A),$$

where

$$\text{Adj}(A)_{ji} = (-1)^{i+j} \det A_{i' \neq i, j' \neq j}$$

The Adjugate matrix is the transpose of the minors of the original matrix (cofactor matrix) times a sign. The element i, j of the cofactor matrix is the determinant of the matrix which is created by removing the column i and row j from the original matrix. Let us implement it!

```
def my_minor(A,i,j):
    return A[np.ix_(np.arange(3)!=i,np.arange(3)!=j)]

A = np.arange(9).reshape(3,3)
A
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])

my_minor(A,0,1)
array([[3, 5],
       [6, 8]])

A = np.array([[1,3,1],[1,1,-1],[3,10,5]],dtype=float)
np.linalg.inv(A)
array([[-7.5,  2.5,  2. ],
       [ 4. , -1. , -1. ],
       [-3.5,  0.5,  1. ]])

C = np.zeros_like(A)
for i in range(3):
    for j in range(3):
        C[i,j] = np.linalg.det(my_minor(A,i,j)) * (-1)**(i+j)

C.T/np.linalg.det(A)
array([[-7.5,  2.5,  2. ],
       [ 4. , -1. , -1. ],
       [-3.5,  0.5,  1. ]])

def my_inv(A):
    C = np.zeros_like(A)
    for i in range(3):
        for j in range(3):
            C[i,j] = np.linalg.det(my_minor(A,i,j)) * (-1)**(i+j)
    return C.T / np.linalg.det(A)
```

```
my_inv(A)
array([[ -7.5,  2.5,  2. ],
       [  4. , -1. , -1. ],
       [-3.5,  0.5,  1. ]])
```

Eigen values, eigen vectors

Characteristic vectors and values of a linear transformation:

$$Av = \lambda v$$

These are the vectors unchanged by the linear operation A .

Example:

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Taking the determinant of $A - \lambda I$, where I is the identity matrix, the characteristic polynomial of A is

$$\det(A - \lambda I) = \begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = 3 - 4\lambda + \lambda^2.$$

This gives us the characteristic equation for the eigenvalues:

$$0 = \det(A - \lambda I) = 3 - 4\lambda + \lambda^2.$$

The two eigenvalues are $\lambda_1 = 1$ and $\lambda_2 = 3$. The eigenvectors are obtained by solving the system of linear equations: $(A - \lambda_i I)v_i = 0$, where v_i is the eigenvector corresponding to eigenvalue λ_i of the matrix A . Obviously $\det(A - \lambda_i I) = 0$ as this is how λ_i was determined. So the eigenvectors are determined up to a constant factor. If there are eigenvalues with multiplicity larger than one then we have an eigenspace and eigenvectors can be chosen to be any independent set spanning that space.

The two eigenvectors for this example are determined thus by:

$$\begin{aligned} (A - \lambda_1 I)v_1 &= \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} v_{11} \\ v_{12} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ (A - \lambda_2 I)v_2 &= \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} v_{21} \\ v_{22} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \end{aligned}$$

This gives us $v_1 = (1, -1)$ and $v_2 = (1, 1)$.

So the solution method used in analytical calculations:

1. Characteristic polynomial using determinant
2. Solution of an n -polynomial equation

3. Solution of a set of singular linear equations

Source: Wikipedia

Find the largest eigenvalue

(from <https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/chapter15.02-The-Power-Method.html>)

In some problems, we only need to find the largest dominant eigenvalue and its corresponding eigenvector. In this case, we can use the power method - a iterative method that will converge to the largest eigenvalue. Let's see the following how the power method works.

Consider an $n \times n$ matrix A that has n linearly independent real eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ and the corresponding eigenvectors $v_1, v_2, \dots, v_n$. Since the eigenvalues are scalars, we can rank them so that $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ (actually, we only require $|\lambda_1| > |\lambda_2|$, other eigenvalues may be equal to each other).

Because the eigenvectors are independent, they are a set of basis vectors, which means that any vector that is in the same space can be written as a linear combination of the basis vectors. That is, for any vector x_0 , it can be written as:

$$x_0 = c_1 v_1 + c_2 v_2 + \dots + c_n v_n$$

where $c_1 \neq 0$ is the constraint. If it is zero, then we need to choose another initial vector so that $c_1 \neq 0$.

Now let's multiply both sides by A :

$$A x_0 = c_1 A v_1 + c_2 A v_2 + \dots + c_n A v_n$$

Since $A v_i = \lambda_i v_i$, we will have:

$$A x_0 = c_1 \lambda_1 v_1 + c_2 \lambda_2 v_2 + \dots + c_n \lambda_n v_n$$

We can change the above equation to:

$$A x_0 = c_1 \lambda_1 \left[v_1 + \frac{c_2 \lambda_2}{c_1 \lambda_1} v_2 + \dots + \frac{c_n \lambda_n}{c_1 \lambda_1} v_n \right] = c_1 \lambda_1 x_1$$

where x_1 is a new vector and $x_1 = v_1 + \frac{c_2 \lambda_2}{c_1 \lambda_1} v_2 + \dots + \frac{c_n \lambda_n}{c_1 \lambda_1} v_n$.

This finishes the first iteration. And we can multiply A to x_1 to start the 2nd iteration:

$$A x_1 = \lambda_1 v_1 + \frac{c_2 \lambda_2^2}{c_1 \lambda_1} v_2 + \dots + \frac{c_n \lambda_n^2}{c_1 \lambda_1} v_n$$

Similarly, we can rearrange the above equation to:

$$A x_1 = \lambda_1 \left[v_1 + \frac{c_2}{c_1} \frac{\lambda_2^2}{\lambda_1^2} v_2 + \dots + \frac{c_n}{c_1} \frac{\lambda_n^2}{\lambda_1^2} v_n \right] = \lambda_1 x_2$$

where x_2 is another new vector and $x_2 = v_1 + \frac{c_2}{c_1} \frac{\lambda_2^2}{\lambda_1^2} v_2 + \dots + \frac{c_n}{c_1} \frac{\lambda_n^2}{\lambda_1^2} v_n$.

We can continue multiply A with the new vector we get from each iteration k times:

$$A x_{k-1} = \lambda_1 \left[v_1 + \frac{c_2}{c_1} \frac{\lambda_2^k}{\lambda_1^k} v_2 + \dots + \frac{c_n}{c_1} \frac{\lambda_n^k}{\lambda_1^k} v_n \right] = \lambda_1 x_k$$

Because λ_1 is the largest eigenvalue, therefore, the ratio $\frac{\lambda_i}{\lambda_1} < 1$ for all $i > 1$. Thus when we

increase k to sufficient large, the ratio of $\left(\frac{\lambda_i}{\lambda_1} \right)^k$ will be close to 0. So that all the terms that contain this ratio can be neglected as k grows:

$$A x_{k-1} = \lambda_1 v_1$$

Essentially, as k is large enough, we will get the largest eigenvalue and its corresponding eigenvector. When implementing this power method, we usually normalize the resulting vector in each iteration. This can be done by factoring out the largest element in the vector, which will make the largest element in the vector equal to 1. This normalization will get us the largest eigenvalue and its corresponding eigenvector at the same time. Let's take a look of the following example.

You may ask when should we stop the iteration? The basic stopping criteria should be one of the three: in the consecutive iterations, (1) the difference between eigenvalues is less than some specified tolerance; (2) the angle between eigenvectors is smaller than a threshold; or the norm of the residual vector is small enough.

The method

Iteration:

$$b_{k+1} = \frac{A b_k}{\|A b_k\|}$$

Condition:

1. The starting vector b_k must have a nonzero component in the direction of the eigenvector of the largest eigenvalue v_1
2. λ_1 is significantly larger than the second largest λ_2

numpy version

```
A = np.array([[2,1],[1,2]],dtype=float)
l,v = np.linalg.eig(A)
print(l[0],v[:,0])
print(l[1],v[:,1])

3.0 [0.70710678 0.70710678]
1.0 [-0.70710678 0.70710678]
```

Power method

```
def one_step(A,v):
    v = np.dot(A,v)
    v /= np.linalg.norm(v)
    return v

A = np.array([[2,1],[1,2]],dtype=float)

b = np.random.random(len(A))-0.5
b

array([-0.03387714, 0.24462765])

numiter = 5
bs = np.zeros((numiter,len(b)),dtype=float)
for k in range(numiter):
    b = one_step(A,b)
    bs[k] = b

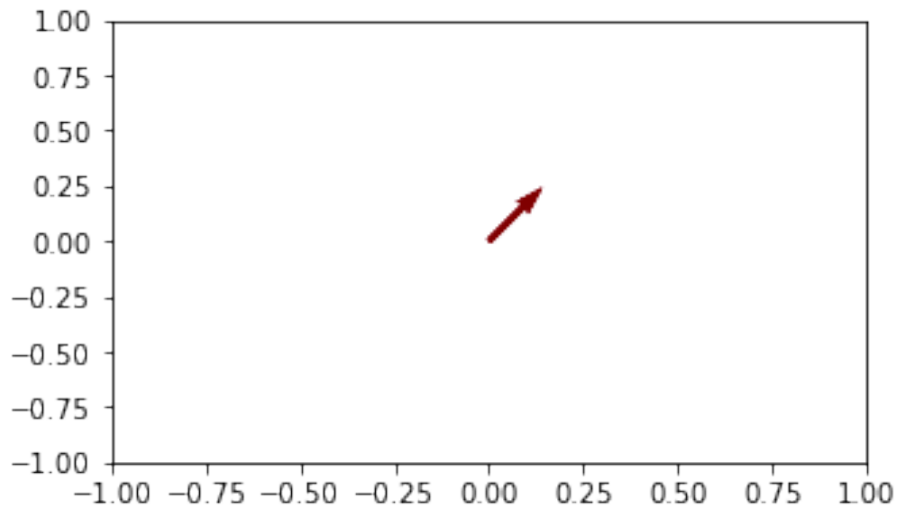
bs

array([[0.70582382, 0.70838742],
       [0.70667938, 0.70753392],
       [0.70696434, 0.70724919],
       [0.70705931, 0.70715425],
       [0.70709096, 0.70712261]])

nulla = np.zeros_like(bs[:,0])
nulla

array([0., 0., 0., 0., 0.])

fig, ax = plt.subplots(figsize =(5, 3))
cmap = cm.jet
ax.axis([-1,1,-1,1])
color = np.dot(bs,bs[-1])
color -= color.min()
color /= color.max()
ax.quiver(nulla,nulla,bs[:,0],bs[:,1],scale=10,color=cmap(color));
```



Eigen values

Obviously should check for zeros....

```
l = (np.abs(np.dot(A,bs[-1])).max()/np.abs(bs[-1]).max())
print(l,bs[-1])
2.9999552418857895 [0.70709096 0.70712261]
np.dot(A,bs[-1]),bs[-1]*3
(array([2.12130452, 2.12133617]), array([2.12127287, 2.12136782]))
```

Second largest Eigenvalue and vector

Shift the spectrum such that the largest will be the smallest

This method can also be used to find very precisely one eigenvalue. Take a value l which close but not equal to the searched eigenvalue λ_i . Shift the spectrum with l then take the inverse matrix. This will make the smallest the biggest and we will get very precisely the deviation from l and the eqigenvalue as well.

```
B = A - np.diag(np.ones(len(bs[-1]))*l)
np.linalg.eig(A),np.linalg.eig(B)
((array([3., 1.]),
  array([[ 0.70710678, -0.70710678],
         [ 0.70710678,  0.70710678]])),
 (array([-8.03744884e-05, -2.00008037e+00]),
  array([[ 0.70710678, -0.70710678],
         [ 0.70710678,  0.70710678]])))
```

```
def power_iter(A,numiter = 5):
    b = np.random.random(len(A)) - 0.5
    bs = np.zeros((numiter,len(b)),dtype=float)
    for k in range(numiter):
        b = np.dot(A,b)
        b /= np.linalg.norm(b)
        bs[k] += b
    l = (np.dot(A,bs[-1])/bs[-1]).mean()
    return (l,bs[-1],bs)
```

```
power_iter(B)
```

```
(-2.000080374488416,
 array([ 0.70710678, -0.70710678]),
 array([[ 0.70713018, -0.70708338],
        [-0.70710678,  0.70710678],
        [ 0.70710678, -0.70710678],
        [-0.70710678,  0.70710678],
        [ 0.70710678, -0.70710678]]))
```

```
A = np.array([[2,1],[1,2]],dtype=float)
B = A - np.diag([1,1])
```

```
np.linalg.eig(B)
```

```
(array([2., 0.]),
 array([[ 0.70710678, -0.70710678],
        [ 0.70710678,  0.70710678]]))
```

The shifted inverse power method

```
A = np.array([[2,1],[1,2]],dtype=float)
B = A - np.diag([2.9,2.9])
```

```
np.linalg.eig(np.linalg.inv(B))
```

```
(array([10.          , -0.52631579]),
 array([[ 0.70710678, -0.70710678],
        [ 0.70710678,  0.70710678]]))
```

```
np.linalg.inv(B)
```

```
array([[4.73684211, 5.26315789],
       [5.26315789, 4.73684211]])
```

```
power_iter(np.linalg.inv(B))
```

```
(10.0000000000001542,
 array([-0.70710651, -0.70710705]),
 array([[ -0.67087809, -0.74156766],
        [-0.7089669 , -0.70524175],
        [-0.70700874, -0.7072048 ]],
```

```
[ -0.70711194, -0.70710162],  
[ -0.70710651, -0.70710705]]))
```

So the eigenvalue is $2.9 + 1/10 = 3$ and the corresponding eigenvector is $(1/\sqrt{2}, 1/\sqrt{2})$.

Multiple largest values

We have a eigenplane and any vector on eigenplane is fine

```
np.linalg.eig(np.array([[1,0],[0,1]]))  
  
(array([1., 1.]),  
 array([[1., 0.],  
        [0., 1.])))  
  
power_iter(np.diag([1,1]))  
  
(1.0,  
 array([ 0.5711944 , -0.82081481]),  
 array([[ 0.5711944 , -0.82081481],  
        [ 0.5711944 , -0.82081481],  
        [ 0.5711944 , -0.82081481],  
        [ 0.5711944 , -0.82081481],  
        [ 0.5711944 , -0.82081481],  
        [ 0.5711944 , -0.82081481]]))
```

Complex eigenvalues

It should not work unless b has the correct complex phase, moreover for real matrices there are always two eigenvalues which are complex conjugate of each other which means that their absolute value is the same.

```
a = np.pi/6  
A = np.array([[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]])  
  
np.linalg.eig(A)  
  
(array([0.8660254+0.5j, 0.8660254-0.5j]),  
 array([[0.70710678+0.j, 0.70710678-0.j],  
        [0. -0.70710678j, 0. +0.70710678j]]))  
  
power_iter(A)  
  
(1.4896218812588902,  
 array([-0.33152444, 0.94344663]),  
 array([[ 0.98281097, -0.18461473],  
        [ 0.94344663, 0.33152444],  
        [ 0.65128653, 0.7588319 ],  
        [ 0.18461473, 0.98281097],  
        [-0.33152444, 0.94344663]]))
```

```

def power_iter_complex(A,numiter = 5):
    b = np.random.random(len(A)) + 1.j * np.random.random(len(A))
    bs = np.zeros((numiter,len(b)),dtype=complex)
    for k in range(numiter):
        b = np.dot(A,b)
        b /= np.linalg.norm(b)
        bs[k] += b
    l = (np.dot(A,bs[-1])/bs[-1]).mean()
    return (l,bs[-1],bs)

power_iter_complex(A,numiter=10)

((0.5889701039725682-0.08926679425746797j),
 array([ 0.62275052+0.59732705j, -0.41472693-0.28876245j]),
 array([[ 0.41472693+0.28876245j,  0.62275052+0.59732705j],
        [ 0.0477888 -0.04858791j,  0.74668124+0.66168163j],
        [-0.33195431-0.37291918j,  0.67053932+0.54873914j],
        [-0.62275052-0.59732705j,  0.41472693+0.28876245j],
        [-0.74668124-0.66168163j,  0.0477888 -0.04858791j],
        [-0.67053932-0.54873914j, -0.33195431-0.37291918j],
        [-0.41472693-0.28876245j, -0.62275052-0.59732705j],
        [-0.0477888 +0.04858791j, -0.74668124-0.66168163j],
        [ 0.33195431+0.37291918j, -0.67053932-0.54873914j],
        [ 0.62275052+0.59732705j, -0.41472693-0.28876245j]])))

C = np.array([[0,-1.j],[-1.j,1.j]])
np.linalg.eig(C)

(array([0.-0.61803399j, 0.+1.61803399j]),
 array([[ 0.85065081+0.00000000e+00j, -0.52573111-3.37723669e-17j],
        [ 0.52573111-0.00000000e+00j,  0.85065081+0.00000000e+00j]]))

power_iter_complex(C,numiter=10)

((5.252395270903113e-05+1.6181566168174313j),
 array([-0.09903492+0.516224j,  0.16019344-0.83548963j]),
 array([[ 0.89673938-0.02149614j, -0.3964132 -0.19558367j],
        [-0.14196805+0.2877439j,  0.12636467-0.93865888j],
        [-0.59687812-0.08035327j,  0.77984985+0.17062847j],
        [ 0.10590789-0.48404729j, -0.15578259+0.85452532j],
        [ 0.52845822+0.09633956j, -0.82780426-0.16183544j],
        [-0.100029 +0.51165819j,  0.15957559-0.83829334j],
        [-0.51810073-0.09862446j,  0.83432712+0.16044661j],
        [ 0.09916165-0.51564352j, -0.16011504+0.83584801j],
        [ 0.51658263+0.09895657j, -0.83526794-0.16024186j],
        [-0.09903492+0.516224j,  0.16019344-0.83548963j]])))

```

Summary

There are other iterative methods to obtain the eigenvalues e.g. QR method which recovers all eigenvalues directly from which the eigenvectors can be obtained using linear algebra. Nevertheless the power method is still used, especially for huge sparse matrices (Page rank of google is a good example)

Change of basis

Basis: $e_1, e_2, \dots, e_n$, here the vectors e_i are linearly independent.

New basis: $f_1, f_2, \dots, f_n$, where the vectors f_i are linearly independent and we know the coordinates of these vectors in the base $\{e_i\}$:

$$\begin{aligned} f_1 &= \sum_{i=1}^n f_{1i} e_i \\ f_2 &= \sum_{i=1}^n f_{2i} e_i \\ &\vdots \\ f_n &= \sum_{i=1}^n f_{ni} e_i \end{aligned}$$

When we talk about a vector or matrix, generally we use its coordinates, in a give basis. The question is if a vector b or matrix A is given in basis $\{e_i\}$ then what are its coordinates in the new basis?

Let us have a vector b in basis $\{e_i\}$, then we can use the coefficients b_i which unambigously determine b .

$$b = \sum_{i=1}^n b_i e_i$$

and

$$b_i = e_i b.$$

The question is what are the $b_i^{(f)}$ coefficients in the bases $\{f_i\}$? We are looking for P such that:

$$b_j^{(f)} = \sum_i P_{ij} b_i^{(e)}$$

It is easy to see that if $b \equiv e_1$, then the first column of P_{ij} is f_1 , and so on, so the columns of P_{ij} are made of the vectors $\{f_i\}$.

$$P = \begin{pmatrix} \begin{pmatrix} f_1 \\ f_1 \end{pmatrix}_1 & \begin{pmatrix} f_2 \\ f_2 \end{pmatrix}_1 & \dots & \begin{pmatrix} f_n \\ f_n \end{pmatrix}_1 \\ \begin{pmatrix} f_1 \\ f_1 \end{pmatrix}_2 & \begin{pmatrix} f_2 \\ f_2 \end{pmatrix}_2 & & \begin{pmatrix} f_n \\ f_n \end{pmatrix}_2 \\ \vdots & \vdots & & \vdots \\ \begin{pmatrix} f_1 \\ f_1 \end{pmatrix}_n & \begin{pmatrix} f_2 \\ f_2 \end{pmatrix}_n & & \begin{pmatrix} f_n \\ f_n \end{pmatrix}_n \end{pmatrix}$$

Example

We use the usual basis for $\{e_i\}$: $e_1 = (1, 0)$ and $e_2 = (0, 1)$.

$$\begin{matrix} f_1 & \textcolor{red}{\hookrightarrow} & (3, 1) \\ f_1 & \textcolor{red}{\hookrightarrow} & (-2, 1) \end{matrix}$$

```
P = np.array([[3, -2], [1, 1]])
P
array([[ 3, -2],
       [ 1,  1]])
```

We transform the vector $(0, 1)_f$ back to the system of $\{e_i\}$:

```
np.dot(P, [0, 1])
array([-2,  1])
A = np.array([[2, 1], [1, 2]])
np.linalg.eig(A)
(array([3., 1.]),
 array([[ 0.70710678, -0.70710678],
        [ 0.70710678,  0.70710678]]))
```

The matrix A in basis $\{f_i\}$:

```
B = np.linalg.inv(P).dot(np.dot(A, P))
B
array([[ 3.4, -0.6],
       [ 1.6,  0.6]])
l, v = np.linalg.eig(B)
l, v
(array([3., 1.]),
 array([[0.83205029, 0.24253563],
        [0.5547002 , 0.9701425 ]]))
```

Eigenvectors of B transformed back to system of $\{e_i\}$.

```
np.dot(P,v[:,0]),np.dot(P,v[:,1])  
(array([1.38675049, 1.38675049]), array([-1.21267813, 1.21267813]))
```